Abstract

A Version Control Systems (VCS), focusing on Git and the collaborative hosting platform GitHub. It traces the development of VCS from local and centralized systems to the current distributed model represented by Git, explaining its basic functions and terminology. The document highlights GitHub's importance as a social coding platform. It shows how it is particularly useful in educational settings for collecting assignments, managing team projects, and encouraging collaboration among students. A comparison of major Git-based hosting platforms, GitHub, GitLab, and Bitbucket, is presented. This section evaluates them based on features, user base, and business use. The report also covers emerging trends and technologies. It discusses AI-powered tools like GitHub Copilot and GitHub Spark, which are influencing the future of developer productivity. Lastly, it looks at the future of VCS, predicting more automation, closer integration with DevOps, and better security measures. The report concludes that Git and GitHub have fundamentally changed digital collaboration. They have become essential tools for reproducible research, modern software development, and technology education.

Keywords: *Version Control System (VCS), Git, GitHub, Distributed Collaboration, GitLab, Bitbucket, Cloud-Based Hosting, Educational Technology, GitHub Copilot*

Table of Contents

Abstract 1
LIST OF FIGURES 3
CHAPTER 1 INTRODUCTION 4
1.1 Background 4
1.1.1 Version Control System 4
1.1.2 Git 4
1.1.3. Github 5
1.1.4. Problem before VCS 5
CHAPTER 2 LITERATURE REVIEW 6
2.1. Virsion Control System (VCS) 6
2.1.1 Local Version Control Systems 6
2.1.2 Centralized Version Control Systems 7
2.1.3 Distributed Version Control Systems 8
2.2. Working mechanism of Git and GitHub 9
2.3 The effectiveness of GitHub cloud services for student 12
2.4 GitHub as a Teaching Tool 13
CHAPTER 3 COMPARATIVE ANALYSIS 14
3.1 A Comparative Analysis of the Rise of Git-Based Distributed Collaborative Hosting Platforms: Survey, Performance Test, and Comparison 14
CHAPTER 4 DISCUSSION ON TOOLS, TRENDS AND TECHNOLOGIES 15
4.1 GitHub Copilot 15
4.2 GitHub Spark 17
CHAPTER 5: CONCLUSION 18
CHAPTER 6: FUTURE SCOPE 18
REFERENCES 19

LIST OF FIGURES

Figure 1 Local version control diagram 7
Figure 2 Centralized version control diagram 7
Figure 3 Distributed version control diagram 8
Figure 4 git add/commit process 9
Figure 5 Working with a local repository 10
Figure 6 Working with both a local and remote repository as a single user 11
Figure 7 Contributing to open source projects 12
Figure 8 Comparative Analysis 15
Figure 9 Github Copilot 16
Figure 10 Github Copilot survey analysis table 17
Figure 11 Github Spark 18

CHAPTER 1 INTRODUCTION

1.1 Background

1.1.1 Version Control System

A Version Control System (VCS) is a specialized program designed to track changes to a file or a specified set of files over time, maintaining a library of all past versions of those files (Beckman et al., 2021). A version control system (VCS) allows you to track the iterative changes you make to your code. Thus, you can experiment with new ideas but always have the option to revert to a specific past version of the code you used to generate particular results. Furthermore, you can record messages as you save each successive version so that you (or anyone else) reviewing the development history of the code is able to understand the rationale for the given edits. It also facilitates collaboration. Using a VCS, your collaborators can make and save changes to the code, and you can automatically incorporate these changes to the main code base. The collaborative aspect is enhanced with the emergence of websites that host version-controlled code (Blischak et al., 2016).

1.1.2 Git

Git is an open-source version control software system (VCS). It is a Distributed Version Control System (Blischak et al., 2016) . Git was originally created by Linus Torvalds in 2005 as a revision management system used for coordinating the development of the Linux kernel. It has since become the leading version control system in software development and scientific research(Spinellis, 2012) .

Box 1. Definitions

• Version Control System (VCS): (noun) a program that tracks changes to specified files over time and maintains a library of all past versions of those files

• Git: (noun) a version control system

• repository (repo): (noun) folder containing all tracked files as well as the version control history

• commit: (noun) a snapshot of changes made to the staged file(s); (verb) to save a snapshot of changes made to the staged file(s)

• stage: (noun) the staging area holds the files to be included in the next commit; (verb) to mark a file to be included in the next commit

• track: (noun) a tracked file is one that is recognized by the Git repository

• branch: (noun) a parallel version of the files in a repository (Box 7)

• local: (noun) the version of your repository that is stored on your personal computer

• remote: (noun) the version of your repository that is stored on a remote server; for instance, on GitHub

• clone: (verb) to create a local copy of a remote repository on your personal computer

• fork: (noun) a copy of another user's repository on GitHub; (verb) to copy a repository; for instance, from one user's GitHub account to your own

• merge: (verb) to update files by incorporating the changes introduced in new commits

• pull: (verb) to retrieve commits from a remote repository and merge them into a local repository

• push: (verb) to send commits from a local repository to a remote repository

• pull request: (noun) a message sent by one GitHub user to merge the commits in their remote repository into another user's remote repository

1.1.3. Github

GitHub is a widely used remote commercial hosting service for Git repositories. It is an online hosting site that provides a dynamic and collaborative environment, and is often referred to as the "social coding platform" (Beckman et al., 2021). GitHub is an online Git service that hosts Git repositories and provides other features such as issue tracking. GitHub has become the prominent platform for hosting open source projects. GitHub has been embraced by the software development community as an important social platform for managing software projects and to support collaborative development (Herwig et al., 2021) .

This seminar report aims to address these questions by examining the effect of GitHub on code velocity through the lens of empirical research and statistical modeling, specifically linear regression. We synthesize findings from multiple recent studies to identify measurable factors and propose a model to quantify their impact. The goal is to move beyond anecdotal evidence and provide a data-driven foundation for optimizing software development workflows on platforms like GitHub.

1.1.4. Problem before VCS

Many scientists write code as part of their research. Just as experiments are logged in laboratory notebooks, it is important to document the code you use for analysis. However, a few key problems can arise when iteratively developing code that make it difficult to document and track which code version was used to create each result. First, you often need to experiment with new ideas, such as adding new features to a script or increasing the speed of a slow step,

but you do not want to risk breaking the currently working code. One often-utilized solution is to make a copy of the script before making new edits. However, this can quickly become a problem because it clutters your file system with uninformative filenames, e.g., analysis.py, analysis_02.py, analysis_03.py, etc. It is difficult to remember the differences between the versions of the files and, more importantly, which version you used to produce specific results, especially if you return to the code months later. Second, you will likely share your code with multiple lab mates or collaborators, and they may have suggestions on how to improve it. If you email the code to multiple people, you will have to manually incorporate all the changes each of them sends (Blischak et al., 2016).

CHAPTER 2 LITERATURE REVIEW

2.1. Virsion Control System (VCS)

A Version Control System (VCS) is a very wise thing to use. It allows you to revert selected files back to a previous state, revert the entire project back to a previous state, compare changes over time, see who last modified something that might be causing a problem, who introduced an issue and when, and more. Using a VCS also generally means that if you screw things up or lose files, you can easily recover.

2.1.1 Local Version Control Systems

Many people's version-control method of choice is to copy files into another directory (perhaps a time-stamped directory, if they're clever). This approach is very common because it is so simple, but it is also incredibly error prone. It is easy to forget which directory you're in and accidentally write to the wrong file or copy over files you don't mean to.

To deal with this issue, programmers long ago developed local VCSs that had a simple database that kept all the changes to files under revision control.

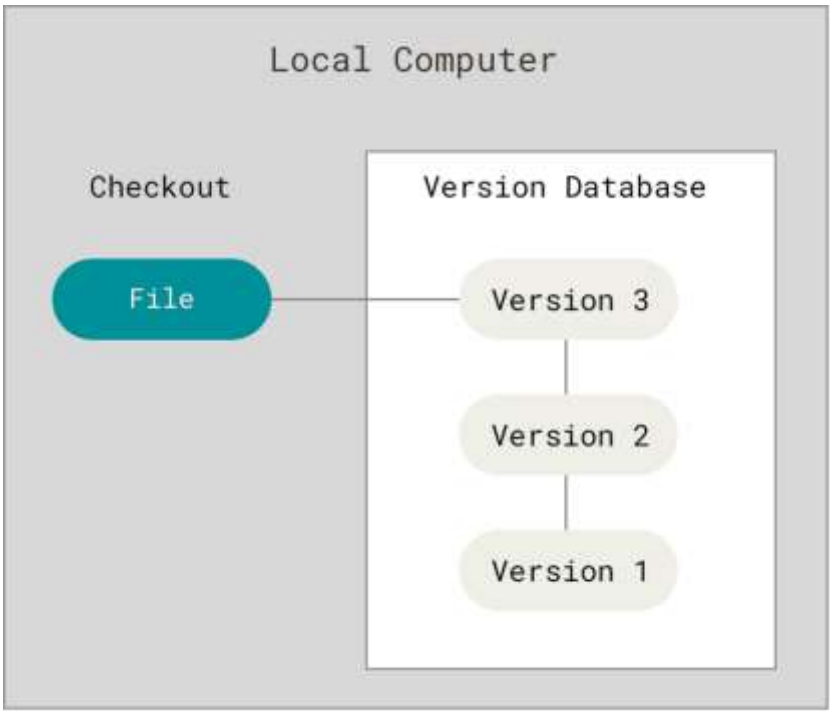

Figure 1 Local version control diagram

One of the most popular VCS tools was a system called RCS, which is still distributed with many computers today. RCS works by keeping patch sets (that is, the differences between files) in a special format on disk; it can then re-create what any file looked like at any point in time by adding up all the patches.

2.1.2 Centralized Version Control Systems

The next major issue that people encounter is that they need to collaborate with developers on other systems. To deal with this problem, Centralized Version Control Systems (CVCSs) were developed. These systems (such as CVS, Subversion, and Perforce) have a single server that contains all the versioned files, and a number of clients that check out files from that central place. For many years, this has been the standard for version control.

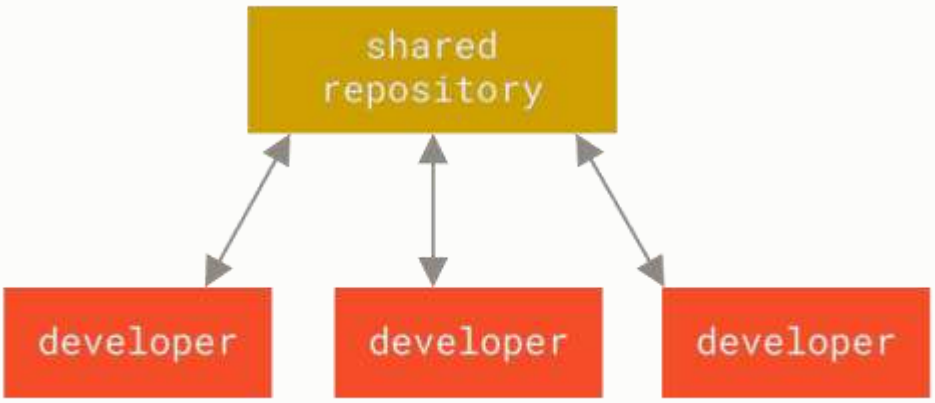

Figure 2 Centralized version control diagram

This setup offers many advantages, especially over local VCSs. For example, everyone knows to a certain degree what everyone else on the project is doing. Administrators have fine-grained control over who can do what, and it's far easier to administer a CVCS than it is to deal with local databases on every client.

However, this setup also has some serious downsides. The most obvious is the single point of failure that the centralized server represents. If that server goes down for an hour, then during that hour nobody can collaborate at all or save versioned changes to anything they're working on. If the hard disk the central database is on becomes corrupted, and proper backups haven't been kept, you lose absolutely everything — the entire history of the project except whatever single snapshots people happen to have on their local machines. Local VCSs suffer from this same problem — whenever you have the entire history of the project in a single place, you risk losing everything.

2.1.3 Distributed Version Control Systems

This is where Distributed Version Control Systems (DVCSs) step in. In a DVCS (such as Git, Mercurial or Darcs), clients don't just check out the latest snapshot of the files; rather, they fully mirror the repository, including its full history. Thus, if any server dies, and these systems were collaborating via that server, any of the client repositories can be copied back up to the server to restore it. Every clone is really a full backup of all the data.

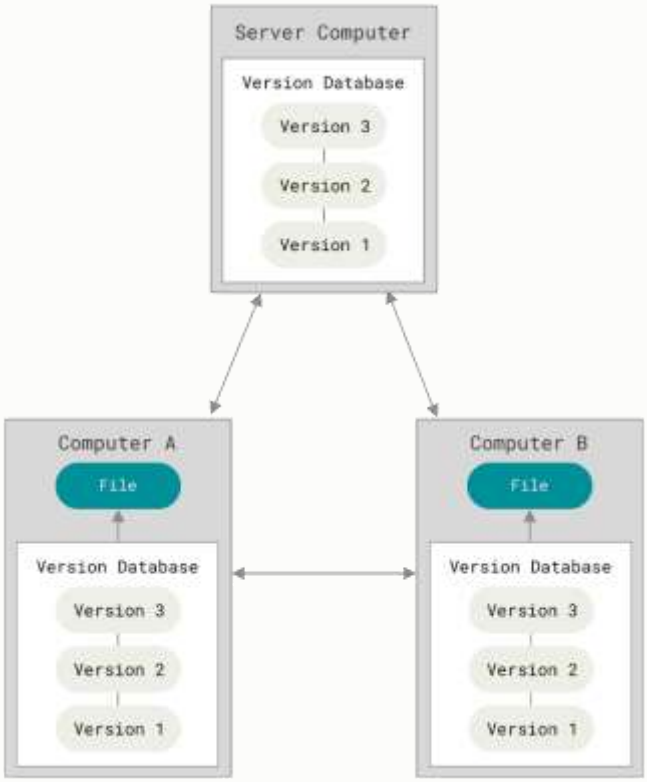

Figure 3 Distributed version control diagram

Furthermore, many of these systems deal pretty well with having several remote repositories they can work with, so you can collaborate with different groups of people in different ways simultaneously within the same project. This allows you to set up several types of workflows that aren't possible in centralized systems, such as hierarchical models (*Git - About Version Control*, n.d.).

2.2. Working mechanism of Git and GitHub

To store a snapshot of changes in your repository, first git add any files to the staging area you wish to commit (for example, you've updated the process.sh file). Second, type git commit with a message. Only files added to the staging area will be committed. All past commits are located in the hidden .git directory in your repository (Blischak et al., 2016).

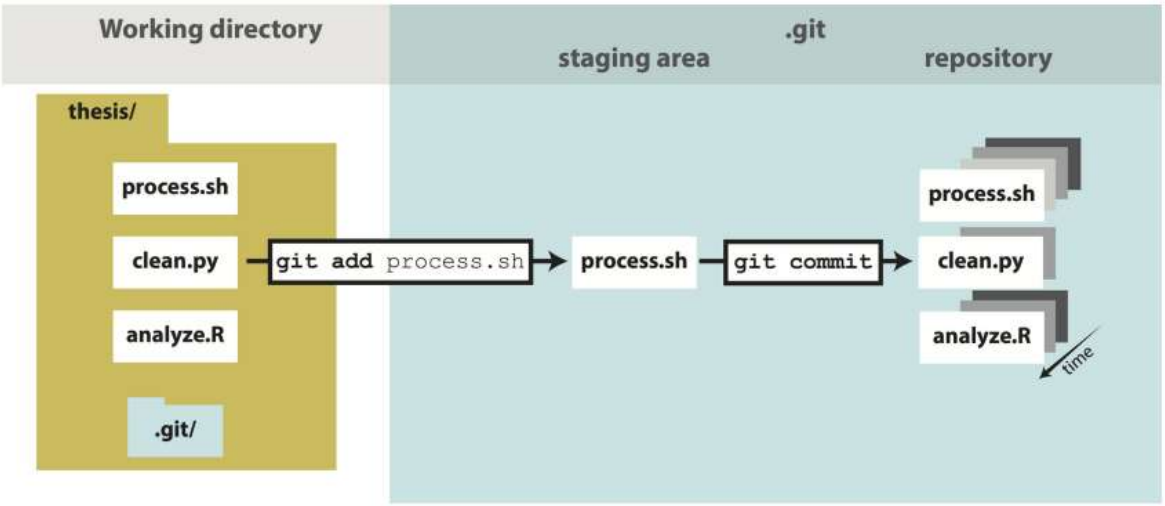

Figure 4 git add/commit process

(A) To designate a directory on your computer as a Git repo, type the command git init. This initializes the repository and will allow you to track the files located within that directory. (B) Once you have added a file, follow the git add/commit cycle to place the new file first into the staging area by typing git add to designate it to be committed, and then git commit to take the shapshot of that file. The commit is assigned a commit identifier (d75es) that can be used in the future to pull up this version or to compare different committed versions of this file. (C) As you continue to add and change files, you should regularly add and commit those changes. Here, an additional commit was done, and the commit log now shows two commit identifiers: d75es (from step B) and f658t (the new commit). Each commit will generate a unique identifier, which can be examined in reverse chronological order using git log (Blischak et al., 2016).

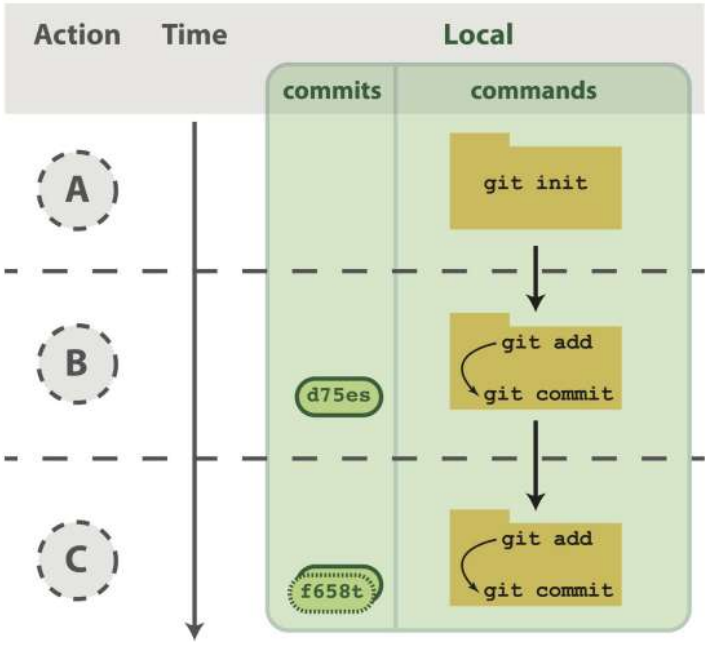

Figure 5 Working with a local repository

(A) On your computer, you commit to a Git repository (commit d75es). (B) On GitHub, you create a new repository called thesis. This repository is currently empty and not linked to the repo on your local machine. (C) The command git remote add connects your local repository to your remote repository. The remote repository is still empty, however, because you have not pushed any content to it. (D) You send all the local commits to the remote repository using the command git push. Only files that have been committed will appear in the remote repository. (E) You repeat several more rounds of updating scripts and committing on your local computer (commit f658t and then commit xv871). You have not yet pushed these commits to the remote repository, so only the previously pushed commit is in the remote repo (commit d75es). (F) To bring the remote repository up to date with your local repository, you git push the two new commits to the remote repository. The local and remote repositories now contain the same files and commit histories (Blischak et al., 2016).

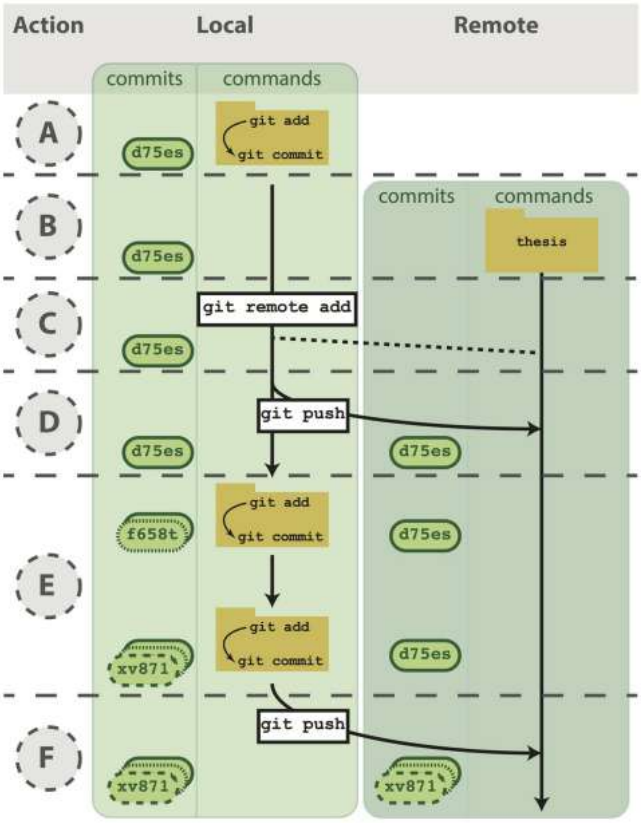

Figure 6 Working with both a local and remote repository as a single user

We would like you to add an empty file that is named after your GitHub username to the repo used to write this manuscript. (A) Using your internet browser, navigate to https://github.com/jdblischak/git-for-science. (B) Click on the “Fork” button to create a copy of this repo on GitHub under your username. (C) On your computer, type git clone https://github.com/username/git-for-science.git, which will create a copy of gitfor-science on your local machine. (D) Navigate to the readers directory by typing cd git-for-science/readers/. Create an empty file that is titled with your GitHub username by typing touch username.txt. Commit that new file by adding it to the staging area (git add username.txt) and committing with a message (git commit -m "Add username to directory of readers."). Note that your commit identifier will be different than what is shown here. (E) You have committed your new file locally, and the next step is to push that new commit up to the git-for-science repo under your username on GitHub. To do so, type git push origin master. (F) To request to add your commits to the original git-for-science repo, issue a pull request from the gitfor-science repo under your username on GitHub. Once your Pull Request is reviewed and accepted, you will be able to see the file you committed with your username in the original git-for-science repository (Blischak et al., 2016).

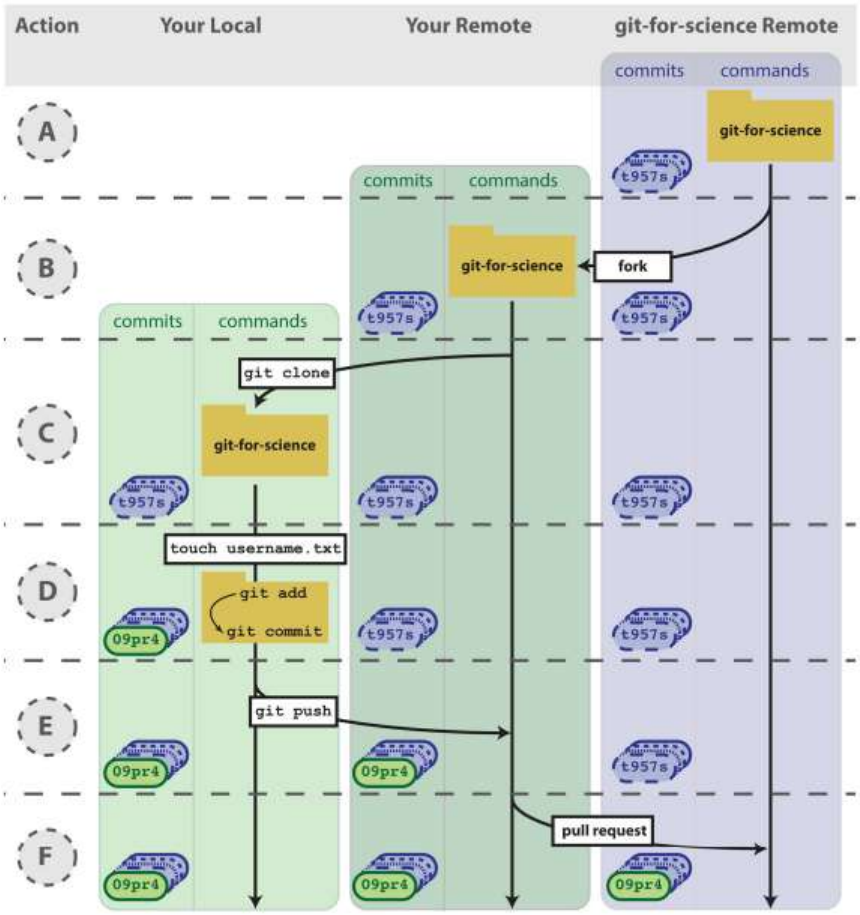

Figure 7 Contributing to open source projects

2.3 The effectiveness of GitHub cloud services for student

In order to determine how effectively GitHub's cloud service enables students to carry out programmatic learning projects, such as interacting with team members, collectively working on code, sharing ideas, and reviewing each other's work during research, students' thoughts and impressions about using this cloud service were gathered. A descriptive study utilizing survey methodology was used as appropriate to achieve the objectives of the study. This allowed the researchers to gain a more detailed view of the students regarding the use of the GitHub cloud service in the execution of educational programming projects using pedagogical observation and peer review methods. The study was conducted among the 3rd year students of the Faculty of Information Technologies of the National University of Life and Environmental Sciences of Ukraine (NUBiP of Ukraine) during the second academic term of 2019 (Herwig et al., 2021).

In order to determine the effectiveness of the cloud service for GitHub collaborative development, 29 students were surveyed on the above-mentioned evaluation indicators after the implementation of the programming educational projects. A questionnaire was developed to ask students to assess the importance of each of the indicators:

i1. Possibility of collaborative development of software;

i2. Ability to manage code versions;

i3. Convenience of bug tracking;

i4. Ability to organize and plan teamwork;

i5. Communication capability;

i6. Ability to support platforms;

i7. Convenience of the code editor;

i8. Security and privacy;

i9. Availability of wiki pages

Table 1. Evaluation of the results of determining the effectiveness of cloud service for GitHub team development.

Indicator	Rank sum	S	Concordance coefficient	Pearson criterion		Sum of converted ranks	Indicator weight
				predictive	table		
i1	35	12100	0,85	197,2	15,5	226	0.22
i2	121	576				140	0.13
i3	62	6889				199	0.19
i4	147	4				114	0.11
i5	173	784				88	0.08
i6	248	10609				13	0.01
i7	96	2401				165	0.16
i8	227	6724				34	0.03
i9	196	2601				65	0.06
∑	1305	42688				1044	1

The concordance coefficient $W = \frac{12*42688}{29^2(9^3-9)}$ = 0.85 indicates a high degree of convergence among Experts The most effective features of this service are the possibility of collaborative development of software (i1), the convenience of bug tracking (i3) and so on.

2.4 GitHub as a Teaching Tool

a) For Collecting Assignments

The most common use of GitHub in education is to collect student programming assignments. Many computer science and software engineering instructors recently adopted GitHub as a host for programming assignments. GitHub is a more efficient platform for storing code than traditional LMS platforms. Students can post assignments easily by dragging and dropping source code files into their repositories and share them with their instructors. They can also modify the code with the built-in editor of GitHub when necessary. Advanced students

typically use features such as branch and merge or perform pull requests. For instructors, management of assignments is an easier task with GitHub. They can access student repositories and see source files with fewer clicks than they do with a typical LMS. Instructors can provide feedback to assignments either directly in the source code using the code editor or typing their comments on the commit window (Angulo & Aktunc, 2019).

b) Collaborating in Team Projects

GitHub is a collaborative platform that can be used by student teams to share and comment on project artifacts. GitHub has been used in thousands of open source projects with success with it social media and collaboration functions [2]. For project or lab assignments, student groups can work on a public repository sharing code and ideas. It also allows cross-team communication with multiple teams working on a larger project as it happens in industry settings or teams exchanging ideas and reviewing each other's' work. GitHub allows the instructor and students to track the contributions of each member so students can be held accountable.

GitHub Classroom with the objective of streamlining the process of publishing and collecting assignments on GitHub . This application serves as a user interface for instructors to publish their assignments in a virtual classroom they create and collect the assignments from student repositories (Angulo & Aktunc, 2019) .

CHAPTER 3 COMPARATIVE ANALYSIS

3.1 A Comparative Analysis of the Rise of Git-Based Distributed Collaborative Hosting Platforms: Survey, Performance Test, and Comparison.

From comparative analysis, we examined market share statistics and identified the major participants in the cloud-based Git repository hosting industry in order to assess the state of the market. With a significant portion of users and repositories, GitHub continues to hold a dominant position. Additionally, GitLab and Bitbucket have sizable user communities, particularly among businesses. Additionally, we looked at the creation of specialized systems that each catered to certain user preferences and needs, such as Gogs, Git-bucket, AWS CodeCommit, Azure DevOps, and Bitbucket Server. This study provides a comprehensive overview of cloud-based Git repository hosting options for academics, professionals, and students. The information presented in this paper should help stakeholders make wise choices, make the most of various platforms, and move ahead

with agility and foresight in the ever changing world of contemporary software development (Uddin et al., 2023).

Feature / Aspect	GitHub	GitLab	Bitbucket
Founding Year	2008	2011	2008
Key Features	Code review, issue tracking, CI/CD, security scanning, GitHub Copilot, etc.	Self-hosting, code review, CI/CD, security scanning, GitLab Pages, etc.	Merge checks, Git LFS, IP whitelisting, Bitbucket Pipelines, pull requests
User Base (as of 2022)	83 million	23 million	5.5 million
Repository Count (as of 2022)	613.3 million	140 million	35 million
CI/CD Integration	Yes (GitHub Actions)	Yes (GitLab CI/CD)	Yes (Bitbucket Pipelines)
Self-Hosting Support	No (only via GitHub Enterprise)	Yes	Yes (via Bitbucket Server/Data Center)
Private Repositories	Yes (since 2011)	Yes (since 2012)	Yes (since 2009)
Pull/Merge Request Features	Advanced (templates, required reviews, WIP)	Advanced (required reviews, WIP)	Basic (no templates, no WIP)
Integration with Other Tools	Extensive	Extensive (especially with Atlassian)	Seamless with Atlassian products
Security Focus	High (security scanning, encryption, access control)	High (security scanning, self-hosting)	Very high (IP whitelisting, two-step verification)
Open Source Community	Very strong	Strong	Moderate
Enterprise Adoption	High (GitHub Enterprise)	High (GitLab Enterprise	High (Bitbucket Enterprise)

Figure 8 Comparative Analysis

CHAPTER 4 DISCUSSION ON TOOLS, TRENDS AND TECHNOLOGIES

4.1 GitHub Copilot

GitHub Copilot is an AI coding assistant that helps you write code faster and with less effort, allowing you to focus more energy on problem solving and collaboration (*What Is GitHub Copilot? - GitHub Docs*, n.d.).

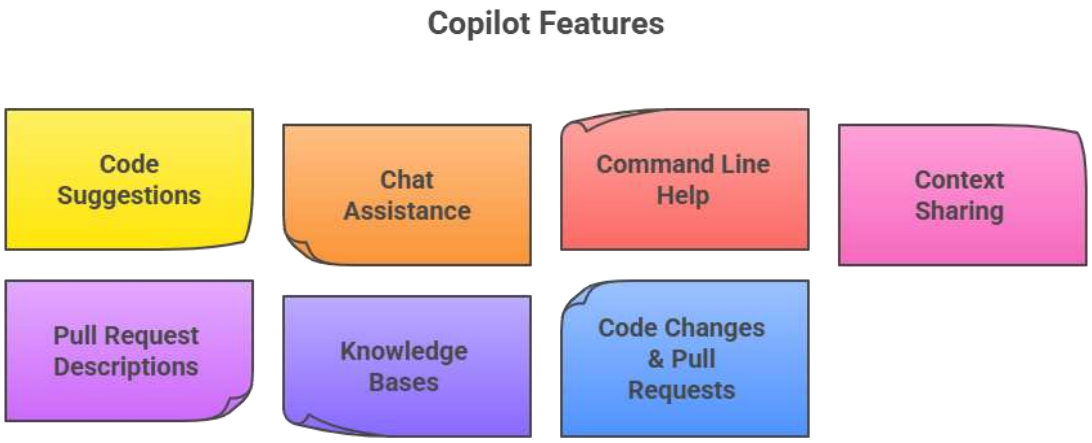

Figure 9 Github Copilot

In a 2021 study, we found that developers' own view of productivity has a twist–it's more akin to having a good day. The ability to stay focused on the task at hand, make meaningful progress, and feel good at the end of a day's work make a real difference in developers' satisfaction and productivity.More than 2,000 developers' survey responses was measuring dimensions of developer productivity when using GitHub Copilot in 2021 found following statistics. *(*Research: Quantifying GitHub Copilot's Impact on Developer Productivity and Happiness - The GitHub Blog*, n.d.)*.

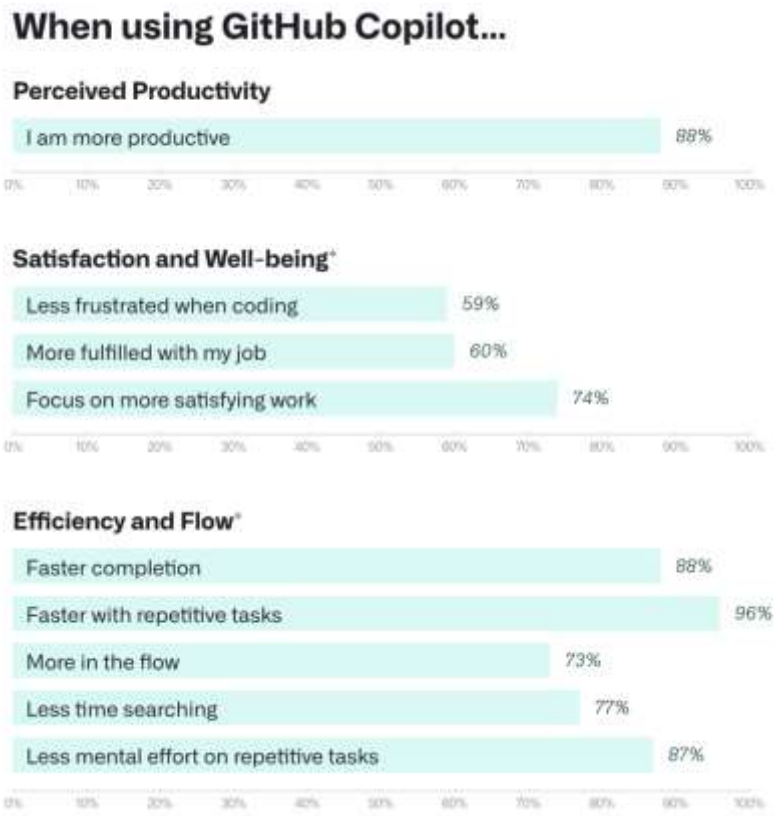

Figure 10 Github Copilot survey analysis table

4.2 GitHub Spark

GitHub Spark, you can describe what you want in natural language and get a fullstack web app with data storage, AI features, and GitHub authentication built in.

You can iterate using prompts, visual tools, or code, and then deploy with a click to a fully managed runtime.Spark is seamlessly integrated with GitHub so you can develop your spark via a synced GitHub codespace with Copilot for advanced editing (*About GitHub Spark - GitHub Docs*, n.d.).

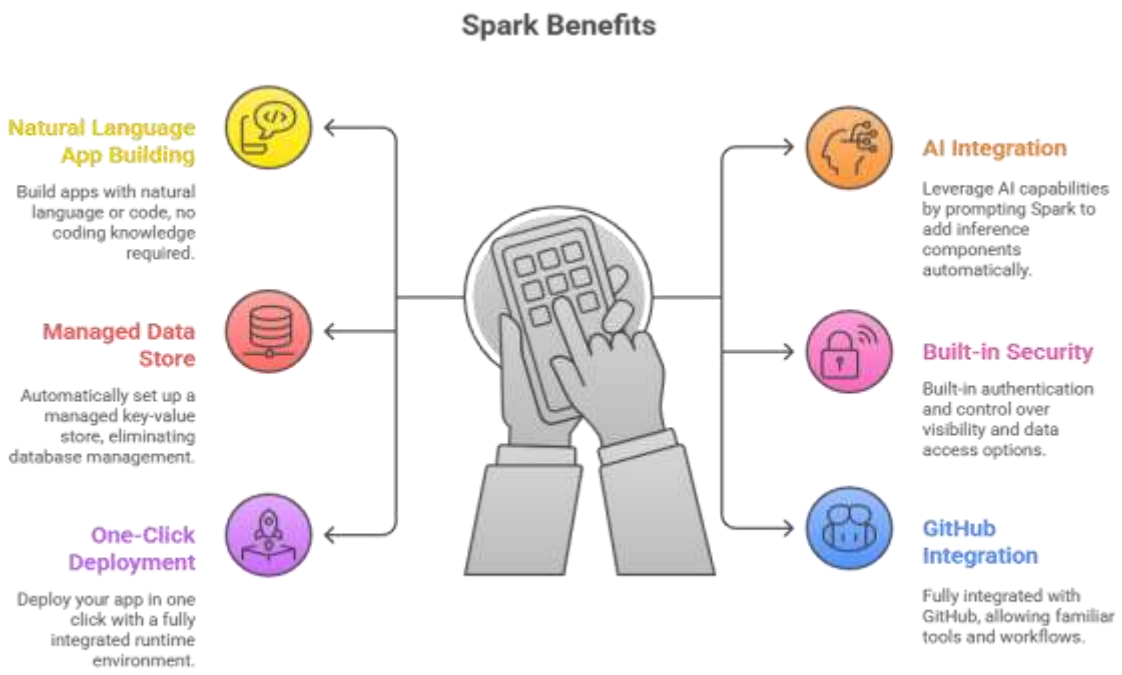

Figure 11 Github Spark

CHAPTER 5: CONCLUSION

The primary conclusion is that Git and GitHub have irrevocably transformed the landscape of digital collaboration and project management, moving the highly efficient, distributed version control model from a specialized software engineering tool to an indispensable foundation for reproducible science and education. Furthermore: Git and GitHub have changed software development, education, and teamwork by creating a simple, distributed version control model that serves as the standard for managing code and digital projects. These tools are being integrated into education, and their growing features, such as AI-assisted coding with GitHub Copilot, deep CI/CD automation, and better security, highlight their key role. They not only help maintain reproducibility and transparency but also shape agile, secure, and innovative workflows for the future.

CHAPTER 6: FUTURE SCOPE

VCS platforms will include more AI-based tools to automatically find bugs, improve code, and suggest changes. The auto-generation of documentation, commit messages, and pull request descriptions will become more common. Real-time coding collaboration, similar to Google Docs for code, will grow in popularity. Improved conflict resolution tools will use AI and machine learning to merge code more intelligently. IT-based workflows will stay central to DevOps pipelines. More automated tools for deployment, testing, and monitoring will link

directly with repositories. AI-driven vulnerability scanning will be implemented in repositories. Automatic license checks, dependency risk assessments, and secure pipelines will also be part of the process.

REFERENCES

About GitHub Spark - GitHub Docs. (n.d.). Retrieved November 25, 2025, from https://docs.github.com/en/copilot/concepts/spark

Angulo, M. A., & Aktunc, O. (2019). *Using GitHub as a Teaching Tool for Programming Courses*. https://doi.org/10.18260/1-2-370.620-31594

Beckman, M. D., Çetinkaya-Rundel, M., Horton, N. J., Rundel, C. W., Sullivan, A. J., & Tackett, M. (2021). Implementing Version Control With Git and GitHub as a Learning Objective in Statistics and Data Science Courses. *Journal of Statistics and Data Science Education*, *29*(S1), S132–S144. https://doi.org/10.1080/10691898.2020.1848485;REQUESTEDJOURNAL:JOURNAL:UJSE21;WGROUP:STRING:PUBLICATION

Blischak, J. D., Davenport, E. R., & Wilson, G. (2016). A Quick Introduction to Version Control with Git and GitHub. *PLOS Computational Biology*, *12*(1), e1004668. https://doi.org/10.1371/JOURNAL.PCBI.1004668

Git - About Version Control. (n.d.). Retrieved November 20, 2025, from https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Herwig, F., Andrassy, R., Annau, N., -, al, Clark, R. N., Swayze, G. A., Eric, K., Glazunova, O. G., Parhomenko, O. V, Korolchuk, V. I., & Voloshyna, T. V. (2021). The effectiveness of GitHub cloud services for implementing a programming training project: students' point of view You may also like Cyberhubs: Virtual Research Environments for Astronomy Imaging Spectroscopy: Earth and Planetary Remote Sensing with the…. *Journal of Physics: Conference Series*, *1840*, 12030. https://doi.org/10.1088/1742-6596/1840/1/012030

Research: quantifying GitHub Copilot's impact on developer productivity and happiness - The GitHub Blog. (n.d.). Retrieved November 21, 2025, from https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/#table-survey-responses-measuring-dimensions-of-developer-productivity-when-using-github-copilot

Spinellis, D. (2012). Git. *IEEE Software*, *29*(3), 100–101. https://doi.org/10.1109/MS.2012.61

Uddin, Md. S., Hasan, K. J., & Tasnima, R. (2023). A Comparative Analysis of the Rise of Git-Based Distributed Collaborative Hosting Platforms: Survey, Performance Test, and Comparison. *Asian Journal of Engineering and Applied Technology*, *12*(2), 29–33. https://doi.org/10.51983/AJEAT-2023.12.2.3969

What is GitHub Copilot? - GitHub Docs. (n.d.). Retrieved November 21, 2025, from https://docs.github.com/en/copilot/get-started/what-is-github-copilot